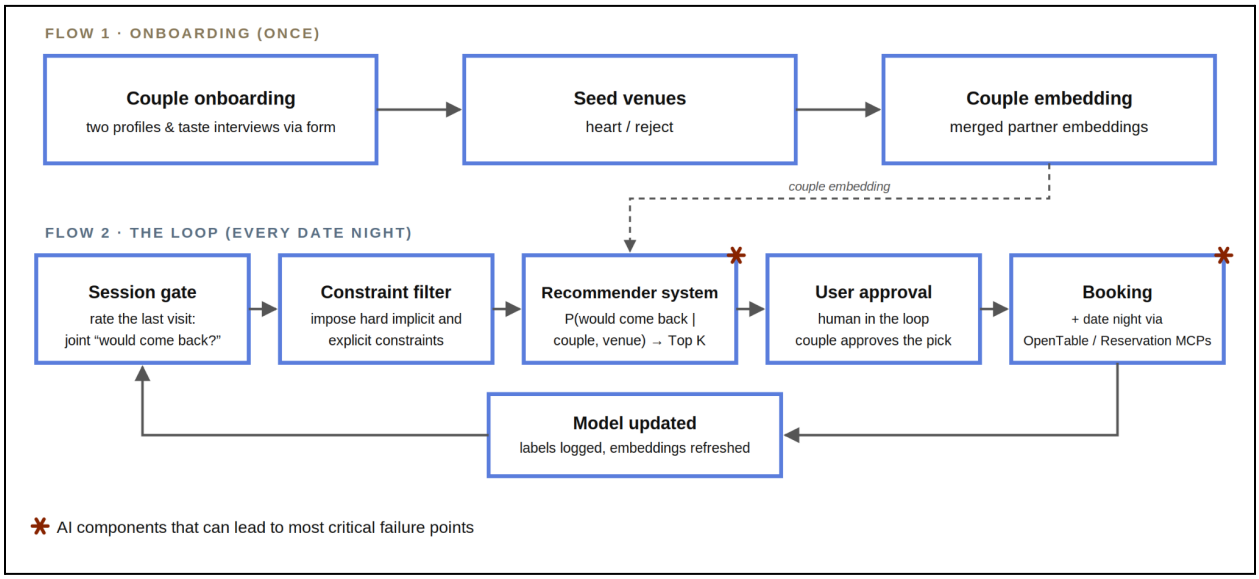
BUSN 43120 81,01 Group 22
Arnav Gulati, Conor O'Hara, Matt Nagler, Robin Zhao

Assignment 3: Technical Solution Flow & Feasibility Risk

0. Updated Value Proposition

For someone in a stable, long-term relationship who struggles to **research and coordinate an experience** both partners enjoy, our AI date-planning assistant uses their preferences to curate a date venue-choice with human approval. Across reviews, social media, and booking platforms, it brings **curation, coordination and excitement** into one process, **reducing planning effort** while helping both parties feel **thoughtfully considered**.

1. Solution Flow Diagram



2. Performance Requirements

Question	Answer
What are the key performance requirements for the AI component of your product?	• **Ranking quality:** Based on held-out visits with AUC, and Precision@K checking if loved venues surface in Top K. Needs cross-couple pooled data due to sparse per-couple labels. • **Cold-start quality floor:** Built from onboarding seed ratings with no visit history; quality matters to prevent early churn.

	• **Constraint compliance:** Guaranteed by design as the pre-ranker filter runs first (e.g., within 5 miles, timing). • **Latency:** Scored in 1–2 seconds; complete agent recommendation turn takes a few seconds to keep conversation fluid. • **Booking reliability:** Zero incorrect bookings across venue, date, and party size, enforced by human approval and post-booking verification.
What training data will you need to meet those requirements?	• **Core set:** couple preferences, venue metadata, and joint visitation labels (return decisions). Onboarding supplies initial seed ratings (heart/rejects). Shortlisted venues passed over are logged as weak negatives. A learned scorer becomes credible at a few thousand pooled labels.
Can you leverage existing models and fine-tuning techniques, or do you need to build your own?	• **Leverage existing models:** off-the-shelf LLMs via Agent SDK (e.g., LangChain) and pretrained venue/couple embedding models. • **Custom build:** Only a small scoring model is trained later on frozen embeddings. No foundation models or deep recommenders built due to sparse data.

3. Identifying Failure Points & Key Feasibility Risks

AI Component 1 – Recommender System

Question	**Answer**
AI capability & input / output	• **Inputs:** Couple embedding (onboarding/past ratings) and metadata/embeddings for filtered candidate venues. • **Outputs:** A ranked Top-K with predicted return probability and short matching rationale from the agent.
Type of model	A recommender system model, predicting P(return | venue embedding, couple embedding).
Objective function & target variable	• **Target Variable:** Binary joint rating ("would both partners return?"). • **Objective Function:** Maximize the likelihood of the joint return outcome based on couple and venue embeddings.

Training data requirements	Joint-visit labels, onboarding seed ratings (heart/reject), and shortlisted venues passed over logged as weak negatives. A learned scorer becomes credible at a few thousand pooled labels across couples.
Failure points	• **Label sparsity and noise** (polluted by mood, weather, or one-off experiences). • **Selection bias**, as labels are only received for system-recommended venues.
AI performance metrics	Precision@k on held-out visits; AUC; and probability calibration.
System performance metrics	**Latency** (end-to-end, from the invocation to the surfacing of options) and **constraint-adherence** (timing, proximity).
Behavior under uncertainty	**Fall back** to safe, high-precision picks (popular or historical favorites), showing fewer high-confidence options.
Costliest error	A **confident poor recommendation** on a high-stakes night (e.g., anniversary), possibly leading to **churn!**
Error handling	**Multi-layered fallbacks:** if the learned scorer fails or times out, fall back to simple analytics-based ranking using historical visits.
Feasibility risks	**Fragile label-collection pipeline:** skipped ratings starve training data, resulting in generic recommendations and eroding the data moat.

AI Component 2 – Agentic Booking System

Question	**Answer**
AI capability & input / output	• **Inputs:** The user constraints, followed by a selected booking recommendation. • **Outputs:** A completed, verified reservation meeting all constraints.
Type of model	An agent powered by a frontier LLM.
Objective function & target variable	• **Target Variable:** Successful reservation completion matching the approved constraints.

	• **Objective:** Minimize execution steps while maximizing reservation success rate and strict constraint adherence.
Training data requirements	No training data is required in order to complete this task, as these are based on pre-trained frontier LLMs.
Failure points	• **Platform failure:** the restaurant is not on a bookable platform, or the booking tool (OpenTable / Reservation MCP) is unavailable, rate-limited, or errors mid-flow**.** • **Stale availability:** the slot shown when the recommendation was made is taken or not available (i.e., restaurant closed) when the agent acts. • **Constraint drift:** the agent books a near-match (different time or party size) when the exact slot is unavailable instead of stopping. • **Payment and account issues:** missing platform login, or a stored payment method declined on a deposit hold.
AI performance metrics	**Correct escalation rate** (end-to-end booking recommendation to accurate booking).
System performance metrics	**End-to-end latency**, **duplicate booking rate**, and **total cost ($) per completed booking** (on the token / LLM side).
Behavior under uncertainty	The only uncertainty might be if the **reservation that was proposed is now booked**. In this scenario, the agent should transparently flag this to the couple and not book, autonomously. The agent could also recommend an alternative based on the chosen recommendation.
Costliest error	**Making an incorrect or duplicate reservation** (wrong date, time, or party size) with a non-refundable deposit or cancellation fee.
Error handling	**Human-in-the-loop approval** before final booking execution, post-reservation state verification via confirmation reading.
Feasibility risk	**Platform access and integration:** OpenTable and Resy do not offer a public third-party booking API, so the build depends on an MCP wrapper or browser automation, which can break when the platform's pages change and may conflict with platform terms of service. **Payment functionality:** For payment-based bookings, there may be a need for payment rails and cached financial credentials, which may cause complexity and concern / lower likelihood-of-use.